Session 06 — Team Work Task

Show / Hide Task List

Time: 45 minutes **Starting point:** your live checkpoint project — filter buttons and search already work.

GOAL

Add one button that hides and shows the entire task list — using the exact same recipe you already used three times today, just applied to something new.

THE RECIPE (you already know this)

1. `useState(...)` → a current value + a setter.
2. A normal, named function → calls the setter.
3. Wire it: `onClick` → the function itself, never `onClick={fn()}`.
4. Read the state back in JSX → a conditional class, or a conditional element.

`TaskItem` is unaffected — you're building on top of it, not changing it.

WHAT YOU NEED TO ADD

1. A new piece of state: `showTasks`, starting as `true`.
2. One normal, named function that flips it.
3. One button, above or below the task list, whose text changes based on `showTasks`.
4. The existing `<ul className="task-list">...</ul>` block, made to exist or not exist based on `showTasks` — using the exact ternary-to-`null` shape already built live for the search feedback.

That's the whole feature. Keep it that small.

USE THESE EXISTING IDEAS

You already watched this exact shape built live today, three times.

- `useState` **gives you a current value and a setter.** Same shape as `currentFilter` and `searchText` — this time the starting value is `true` instead of a string. `useState` works with any kind of value: a string, a number, a boolean — always the same shape: `const [value, setValue] = useState(initialValue)`.
- **A normal, named function calls the setter.** Same shape as `handleShowAll`.
- `!` **flips a boolean.** You already know this operator from Session 04C's own `if (!x)` guards — it reads as "the opposite of whatever this currently is." Work out how to hand the setter the opposite of the current value, rather than a hardcoded `true` or `false`.
- **Wire the function itself, not a call to it.** `onClick={yourFunctionName}`, never `onClick={yourFunctionName()}` — the exact mistake demonstrated live today.
- **A ternary can choose text, or a whole element.** You already built both today: the filter buttons' className ternary chose a string; the search-feedback ternary chose whether a `<p>` existed at all. This teamwork task uses both flavors — one ternary for the button's text, one for the whole task list.

WHAT TO WORK OUT YOURSELVES

Do not copy a finished shape from anywhere — this is the part you derive, using the recipe above:

- A boolean piece of state named `showTasks`, starting `true`. Use the exact `useState` shape you already used live today, just with a different name and a boolean starting value instead of a string.
- One normal, named function that toggles it. It needs to flip `showTasks` to whatever it currently isn't — you already know the operator for that from Session 04C's own `if (!x)` guards.
- The button's visible text should change between "Hide tasks" and "Show tasks" depending on `showTasks` — the same ternary-in-JSX idea already used for the className strings today, just choosing text instead of a class name.
- The task list itself should follow the exact shape already built live for the search-feedback paragraph: a condition, a `?`, the JSX, a `:`, and what React renders when there's nothing to show.

STYLING (PROVIDED — paste this in as-is)

This isn't a CSS lesson, so the button's styling is given to you. Add this to `src/index.css`, anywhere after the `.search-feedback` rule:

```
.toggle-tasks-button {
    background-color: #ffffff;
    border: 1px solid #e2e5ea;
    border-radius: 8px;
    color: #1f2430;
    cursor: pointer;
    font-size: 15px;
    padding: 10px 18px;
    margin-top: 15px;
}

.toggle-tasks-button:hover {
    border-color: #3b6ef5;
}
```

Give your button `className="toggle-tasks-button"`.

HELPFUL REMINDERS

- No `.map()`. No arrow functions. No callback props. No collection state (`showTasks` is one boolean, not an array). If your team reaches for any of these, stop and re-read the GOAL above — none of them are needed.
- `TaskItem` and its three existing calls do not change at all — you're wrapping them, not editing them.
- The filter buttons and search box keep working exactly as they already do — this is a completely separate, additional piece of interactivity, not a change to filtering.
- Make sure the button's own label actually changes when clicked — that's how you'll know the ternary is reading `showTasks` correctly.

ACCEPTANCE CRITERIA

Your team is done when:

1. `showTasks` exists as `useState(true)` inside `App`.
2. A normal (not arrow) named function toggles it using `!showTasks`.
3. One button is wired with `onClick={theFunctionItself}` — no parentheses.
4. The button's text reads "Hide tasks" when the list is visible, and "Show tasks" when it isn't.

5. The `<ul className="task-list">` block only renders when `showTasks` is `true`; otherwise nothing renders in its place.
6. Clicking the button repeatedly toggles the list and the button's own label correctly, every time.
7. The filter buttons and search box (with its "Searching for" feedback) still work exactly as before.
8. `npm run build` succeeds with no errors.
9. No `.map()`, arrow functions, callback props, or collection state appear anywhere in your solution.

TEST CASES

- Run `npm run dev`. Confirm the button initially reads "Hide tasks" and the three task rows are visible.
- Click the button: the three task rows disappear, and the button now reads "Show tasks".
- Click it again: the three task rows reappear, and the button reads "Hide tasks" again.
- Confirm the filter buttons still highlight correctly, and the search box still shows/hides its "Searching for" feedback correctly, unaffected by the new button.
- Confirm `npm run build` completes with **zero** TypeScript errors.